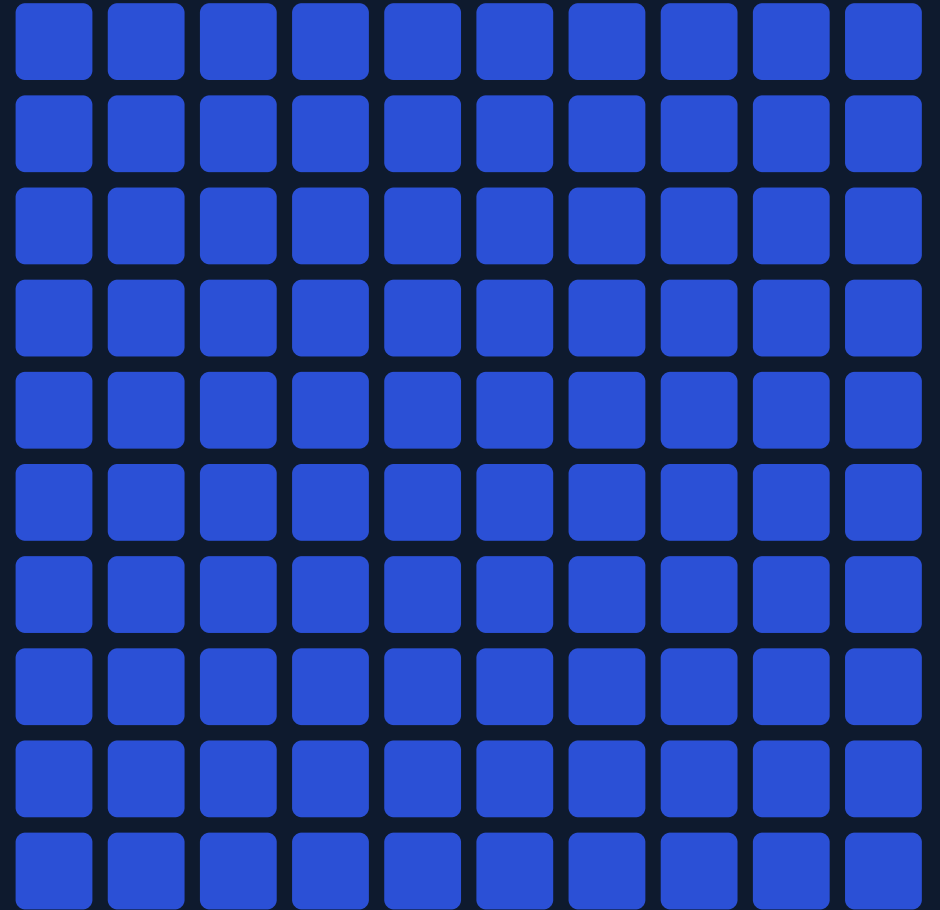


SALESTORM

Keeping every transaction correct when 10,000
customers compete for 100 units

Team NullPointer · SYSCRAFTERS 2026 design-first hackathon



100 / 100 sold 0 oversold · 0 double charges

10,000 buyers, 100 units, one second

0:00 – 0:30 · Problem

10,000

customers press Buy now

100

units in stock

1 s

burst window

The business question

How can we handle thousands of simultaneous purchase requests for limited inventory without overselling, while keeping payment and order processing reliable?

What goes wrong in a naive design



Oversell

Two buyers both get the last unit. We cancel a paid order or ship stock we do not have.



Double charge

A double click or a gateway timeout charges the same card twice.



Lost order

Payment is taken while the Order Service is down, and no order is ever created.

Five guarantees we never break

0:30 – 1:00 · Requirements



Never sell more than stock

Units sold $\leq$ units in stock, at every instant



Stock always adds up

available + reserved + sold = stock



One reservation per request

Per idempotency key, and one active hold per customer



One charge per reservation

The same key at our database and at the payment gateway



Every payment ends well

Exactly one confirmed order, or a refund

Targets, monitored but not guaranteed

≤ 200 ms

reserve p99, server side

≤ 50 ms

sold-out answer p99, from Redis

≤ 5 s

order confirmed after payment (p95)

99.95 %

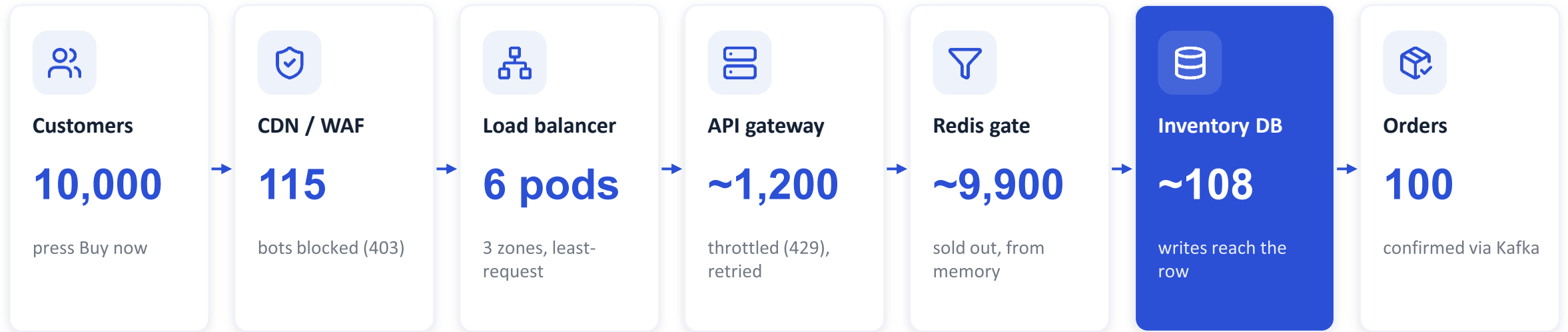
buy-path availability in the sale window

500k req/s

flash-sale peak absorbed at the edge

Each layer sheds load before the database

1:00 – 2:00 · HLD



Measured in our 10,000-customer simulation. The database is the only consistency point; Redis is a filter.



Synchronous: decisions

Reserve, start payment, charge. The customer needs the answer now, so these are request/response calls.



Asynchronous: consequences

Confirm stock, create the order, fulfil, notify. Events go through a transactional outbox and Kafka.



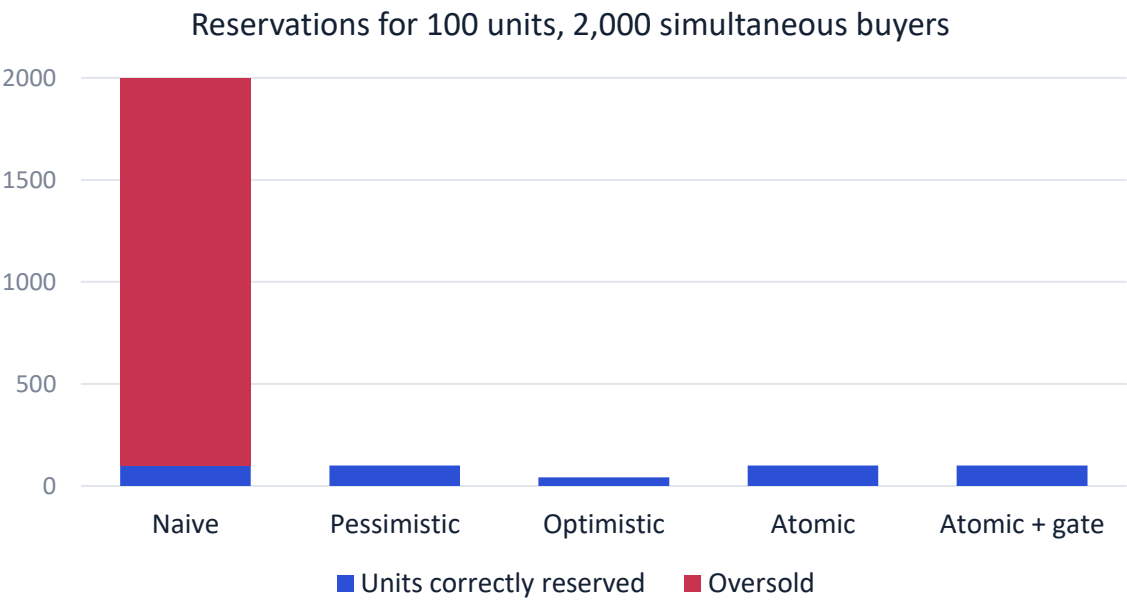
Database per service

Inventory, payment and order each own their data. Boundaries follow consistency boundaries.

One SQL statement stops overselling

```
UPDATE inventory
  SET available = available - 1,
      reserved = reserved + 1
 WHERE available >= 1;
```

-  Two buyers, last unit: the row lock serialises them. The second re-checks the guard, updates 0 rows and gets SOLD_OUT.
-  The Redis gate stops about 99 % of requests before they reach the row.
-  CHECK (available >= 0) is the backstop if any code path is ever wrong.



Approach	Correct?	p99	DB calls
Naive	Oversold 1,900	~41 ms	4,000
Pessimistic	Yes, slow	~4,450 ms	2,100
Optimistic	Sold only 43	~140 ms	~27,800
Atomic + gate	Yes, selected	~3 ms	100

Every payment ends in an order or a refund

3:00 – 3:45 · Payment & order



One idempotency key per reservation, `pay_{reservationId}`, checked at our database and at the payment gateway.
Double clicks and retries can never charge twice.



Payment succeeds

Payment row and PaymentSucceeded are committed in one transaction (outbox), then the order is confirmed.

Order CONFIRMED



Payment declined

PaymentFailed releases the unit straight back to stock, so the next customer can buy it.

Unit RELEASED



Gateway times out

Status is UNKNOWN, never guessed. The reconciler asks the gateway by key and records the real result.

No second charge



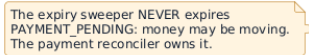
Order Service down

The event waits in Kafka and the consumer retries. When the service recovers, the order confirms.

100 / 100 after a 30 s outage

3:45 – 4:30 · LLD & SOLID

State Diagram – Inventory Reservation



Fails fast when the PSP is down

At 50× traffic, only the edge grows

4:30 – 5:00 · Scale & proof

Layer	10k req/s	500k req/s
CDN / WAF	Easy	Primary shield
Waiting room	Idle	Admits ~20k users/s
API gateway	~10 pods	~60–100 pods
Redis gate	~10k ops/s	Bucketed keys if needed
Inventory DB	~110 writes	~110 writes
Payment / Order	≤ 100 each	≤ 100 each

Load after the gate is bounded by stock, not by traffic.

Only stateless layers scale out. The database, payments and orders see the same load at 10k or 500k req/s.



15 / 15

automated tests pass, incl. one that must fail



8 / 8

guarantees hold, checked live



0

requests sent to a dead gateway pod



0

double charges across 10,000 customers

Prototype built with AI assistance (Claude). Every module was reviewed against our design, tested, and traced against our sequence diagrams.

100 units, 10,000 clicks: what happens, step by step

1



Edge

CDN / WAF filters bots. A waiting room hands out signed queue tokens if admission exceeds capacity.

2



Gateway

JWT checked, request validated, token bucket applied. Excess gets 429 and retries with the same key.

3



Idempotency and limit

Double clicks collapse into the first request. One active hold per customer.

4



Redis gate

The first ~100 take a token. Everyone else hears "sold out" from memory in about a millisecond.

5



Database guard

Conditional UPDATE, reservation and outbox event in one transaction. The last unit goes to exactly one buyer.

6



Pay, then orders

Payment by key through a circuit breaker. Events confirm stock and orders. Sold ≤ 100 , every payment has one order.

Thank you

Every transaction correct, under any load, through any failure.

Questions welcome. Live dashboard and full blueprint in the submission folder.

Team NullPointer



System architect

HLD, scalability, deployment



LLD & design engineer

LLD, SOLID, UML, patterns



Data & API engineer

Database, APIs, events



Reliability engineer

Concurrency, failures, security